

ATIVIDADE

Assunto:

Interfaces.

Orientações:

A atividade deve ser executada individualmente e entregue através do ambiente *Google Classroom*.

Regras de criação dos programas:

Crie um novo projeto Java denominado **AtividadeInterfaces**. As classes devem possuir os nomes informados no texto. Ao final, o projeto deve ser exportado para um arquivo em formato ZIP.

Nome completo:

Hosana Fernandes Gomes

1. Quais as diferenças entre classes abstratas e interfaces? Explique.

No Java, uma classe só pode herdar de uma ÚNICA classe abstrata porque não é permitido herança múltipla de classes. Mas essa mesma classe pode implementar várias interfaces. Uma classe só pode herdar de uma única classe abstrata (extends), pois o Java não permite herança múltipla de classes. Já uma classe pode implementar infinitas interfaces, o que deixa o sistema mais flexível para se adaptar às necessidades de negócio. Quanto aos métodos e parâmetros, as classes abstratas podem declarar conforme necessidade, mas, na interface, todos os métodos e parâmetros são NECESSARIAMENTE public static final, isto é, constantes e imutáveis.

2. Interfaces podem ter métodos concretos? Explique.

Sim, são os default methods

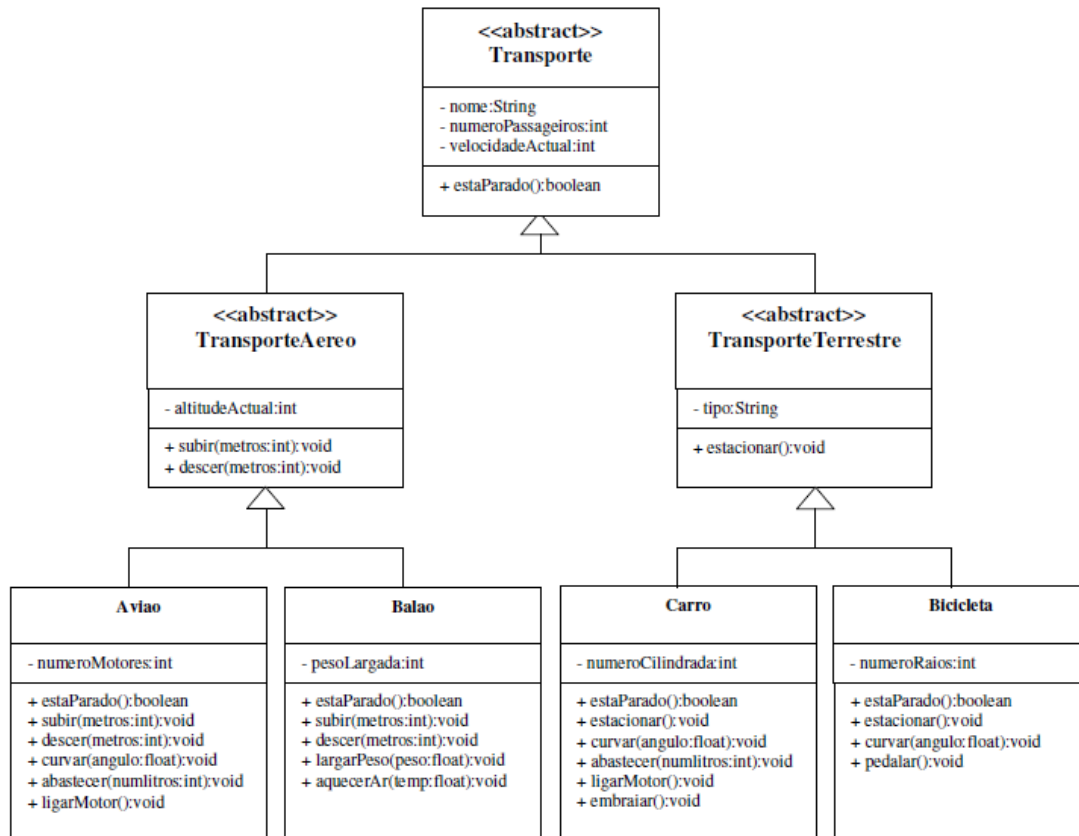
3. Demonstre como o uso de *default methods* pode evitar a repetição de código.

Eles permitem que você escreva um comportamento padrão diretamente na interface. Assim, todas as classes que implementarem essa interface vão herdar esse código automaticamente, sem a necessidade de duplicar a mesma lógica em dezenas de classes diferentes.

4. Em uma situação em que classes abstratas e interfaces são opções viáveis, qual deve ser utilizada prioritariamente?

A recomendação atual da engenharia de software é usar Interfaces prioritariamente: (1) preserva a “herança única”, evitando que a mesma seja utilizada indevidamente em alguma classe; (2) mais fácil de realizar testes unitários. Depois de conter os default methods, as interfaces conseguem fazer quase tudo o que uma classe abstrata faz em termos de compartilhamento de código, dispondo das duas vantagens supracitadas.

5. Considere o diagrama UML a seguir e faça o que se pede:



O que se pede:

- Crie uma interface de nome Motorizado em que são declarados os métodos void ligarMotor() e void abastecer(int numLitros).
- Implemente a interface Motorizado nas classes Aviao e Carro.
- Escreva um programa de teste capaz de verificar a implementação anterior.
- Crie uma interface de nome Conduzivel onde é declarado o método void curvar(float angulo).
- Implemente a interface Conduzivel nas classes Aviao, Carro e Bicicleta.
- Complete o programa de teste criado anteriormente por forma a testar estas últimas implementações.

Boa sorte!

Prof. Igor.